E0 297: Cryptographic Hardware Accelerators

Design Assignment 3: RSA-2048 Modular Exponentiation

Anirudh Chaturvedi
SR No: 25850

November 12, 2026

1 Introduction & Design Methodology

This report presents the RTL design, microarchitecture, control/datapath state machines (FSM, FSMD, ASM, ASMD), design truth tables, functional verification, and ASIC synthesis of a parameterized RSA-2048 modular exponentiation core (`modexp #(.N(2048))`) computing

$$\text{modexp_out} = \text{inp}^{\text{exp}} \bmod P$$

using a **Left-to-Right Binary Square-and-Multiply** controller paired with a **Simple Interleaved Modular Multiplier** (`modmul`). The design is fully parameterized by bit-width N and uses no optimizations specific to the RSA-2048 modulus P, making it applicable to any N-bit modulus.

1.1 Top-Level Interface & Input Pre-Reduction (`modexp.v`)

The top-level module preserves the exact required port list (`clock`, `write`, `reset` active-low synchronous, `start`, `ready`, `inp[N-1:0]`, `exp[N-1:0]`, `modexp_out[N-1:0]`) and hardcoded 2048-bit modulus P:

- **Input Latching (`write`):** On the rising clock edge where `write` is asserted, `inp` and `exp` are latched into `inp_reg` and `exp_reg`.
- **Modular Pre-Reduction (`start`):** Since Test Vectors 2, 4, and 5 in `inp_2048.txt` have inp $\geq P$, `base` is reduced on `start` via `(inp_reg >= P) ? (inp_reg - P) : inp_reg`. Because the MSB of P is 1, inp $< 2^N < 2P$, so a single conditional subtraction guarantees $0 \leq$ base $< P$.
- **Square-and-Multiply Controller:** Scanning `exp_shift` from MSB $(N-1)$ down to LSB (0), each bit performs res $\leftarrow$ (res$\times$res) mod P, followed by res $\leftarrow$ (res$\times$base) mod P only if `exp_shift[N-1] == 1`.

1.2 Simple Interleaved Modular Multiplier (`modmul.v`)

Following Lecture 17 (Slide 11), `modmul` computes $(A \times B) \bmod P$ in N clock cycles for $0 \leq A, B < P$:

1. Initialize $(N+2)$-bit accumulator `acc` $\leftarrow 0$ and shift register `a_reg` $\leftarrow A$.
2. Each cycle, compute `acc_shifted` $= 2 \cdot$ `acc` $+$ (`a_reg`$[N-1]$? $B : 0$), bounded by $3P$ ($N+2$ bits).
3. Reduce modulo P in one combinational step: subtract $2P$ if `acc_shifted` $\geq 2P$, else subtract P if `acc_shifted` $\geq P$, else keep `acc_shifted`.

2 Microarchitecture Block Diagram ($N = 2048$)

Figure 1 illustrates the complete hardware microarchitecture of `modexp` and `u_modmul` using standard textbook datapath conventions: synchronous D-register blocks (with clock-edge triangles), 2:1 operand multiplexers, combinational modular arithmetic units, and dashed control paths.

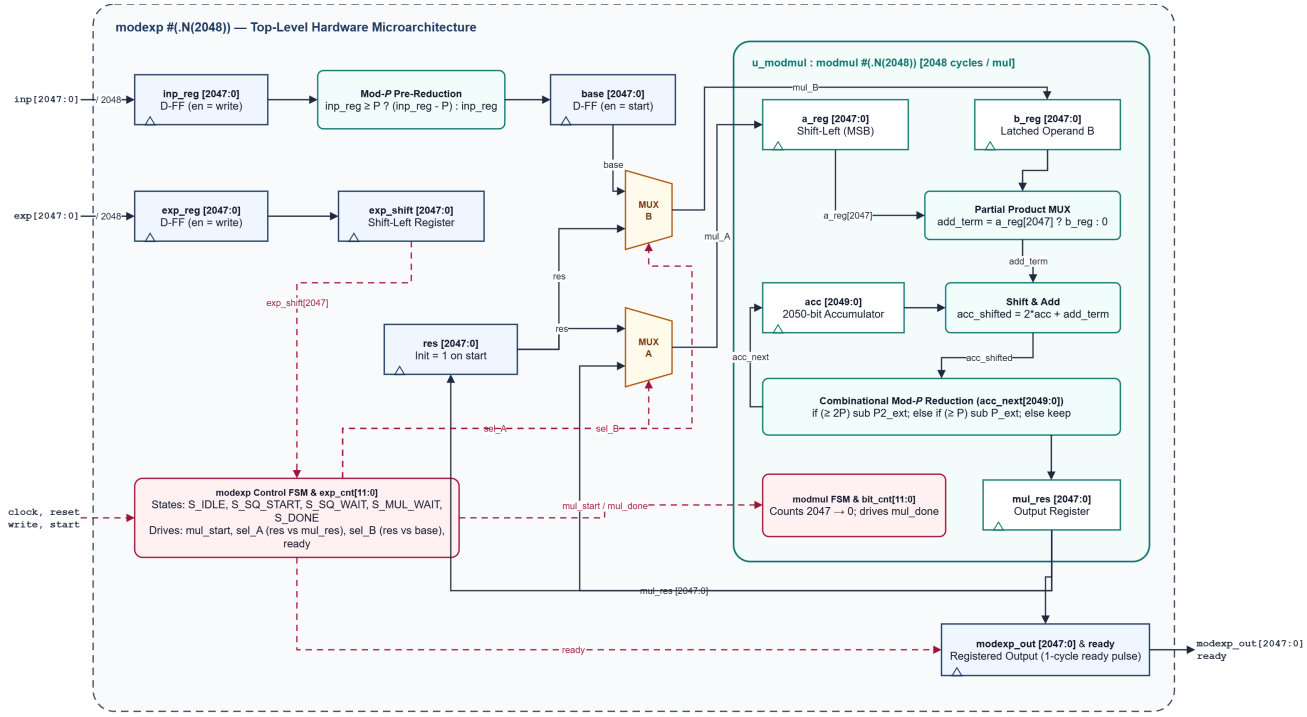


Figure 1: Top-level hardware microarchitecture of `modexp #(.N(2048))` and its internal `u_modmul` interleaved modular multiplier.

3 Control FSM & FSMD Diagrams

Figure ?? presents the Finite State Machine with Datapath (FSMD) for both the 5-state top-level `modexp` controller and the 2-state `modmul` multiplier, showing exact state encodings, Mealy/Moore transitions, and RTL register transfers.

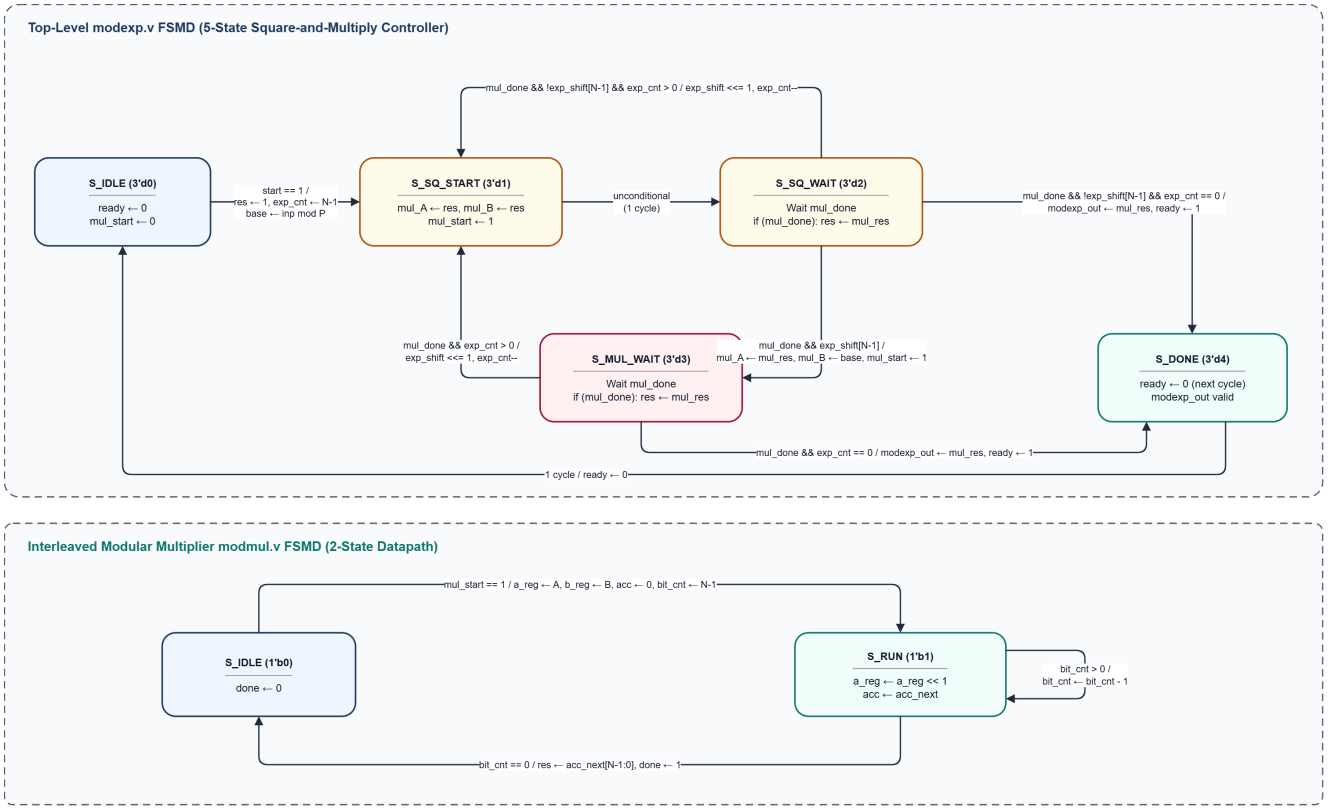


Figure 2: Control FSMD state transition diagrams for `modexp.v` (5 states) and `modmul.v` (2 states).

4 ASM & ASMD Charts

Figure **??** presents the Algorithmic State Machine with Datapath (ASMD) charts using standard hardware notation: rectangular state boxes (navy), decision diamonds (amber), and rounded conditional RTL output boxes (teal).

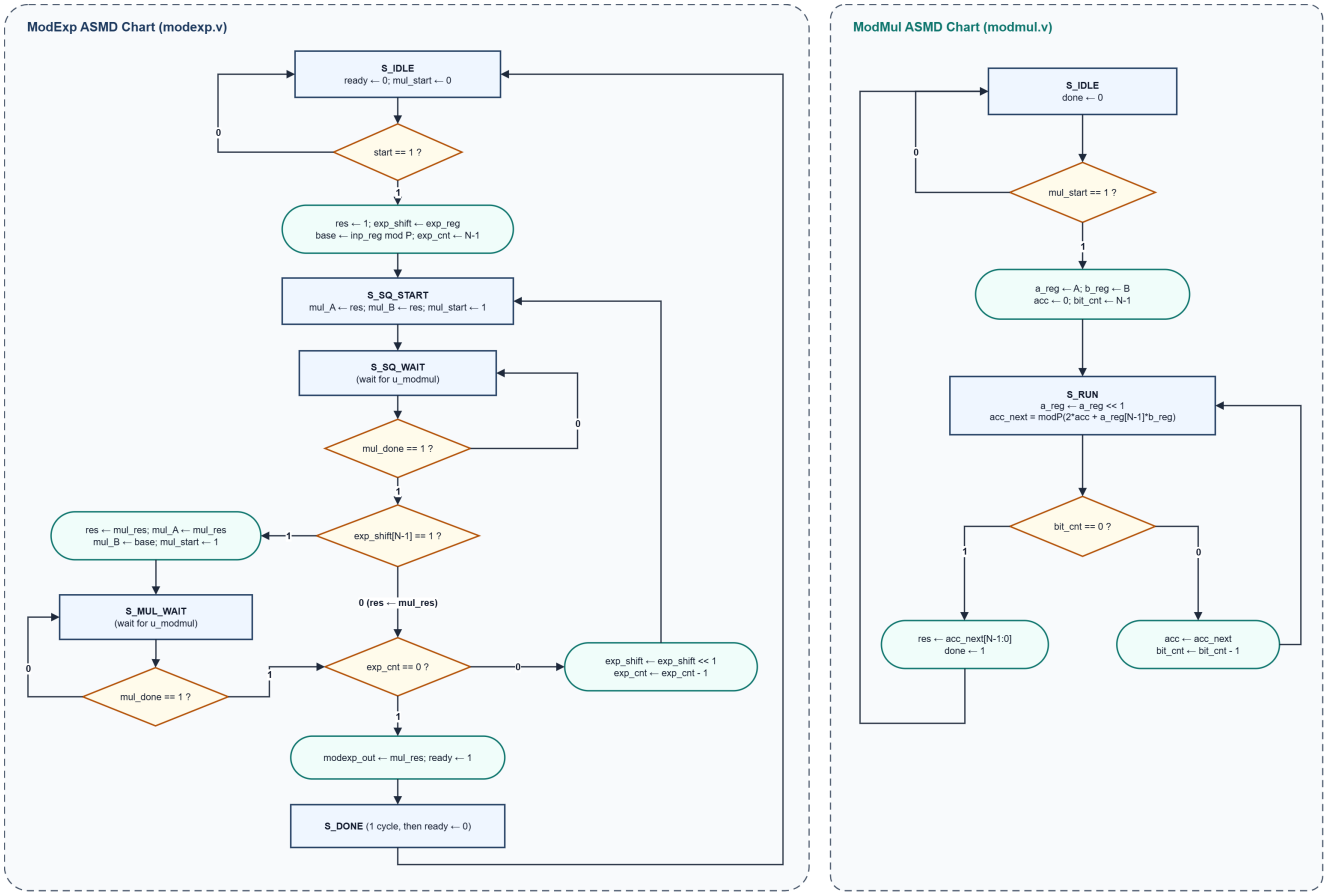


Figure 3: ASMD charts for `modexp.v` (left) and `modmul.v` (right), showing state boxes, decision diamonds, and conditional output ovals.

5 Design Truth & Control Tables

Tables 1, 2, and 3 specify the exact state-transition truth table, combinational modular reduction truth table, and operand multiplexer control logic used to synthesize the RTL.

Table 1: Top-Level `modexp` FSM State-Transition & Output Truth Table

Current State	**Condition**	**Next State**	`mul_start`	`ready`	**Datapath Action**
`ANY`	`reset == 0`	`S_IDLE`	0	0	Clear all regs
`S_IDLE`	`start == 0`	`S_IDLE`	0	0	Hold idle
`S_IDLE`	`start == 1`	`S_SQ_START`	0	0	`res`←1, `base`←`inp_reg` mod P, `exp_cnt`← $N-1$
`S_SQ_START`	*unconditional*	`S_SQ_WAIT`	1	0	`mul_A`←`res`, `mul_B`←`res`
`S_SQ_WAIT`	`mul_done == 0`	`S_SQ_WAIT`	0	0	Wait for `u_modmul`
`S_SQ_WAIT`	`mul_done && exp_shift[N-1]`	`S_MUL_WAIT`	1	0	`res`←`mul_res`, `mul_A`←`mul_res`, `mul_B`←`base`
`S_SQ_WAIT`	`mul_done && !exp_shift[N-1] && exp_cnt>0`	`S_SQ_START`	0	0	`res`←`mul_res`, `exp_shift`≪=1, `exp_cnt--`
`S_SQ_WAIT`	`mul_done && !exp_shift[N-1] && exp_cnt==0`	`S_DONE`	0	1	`res`←`mul_res`, `modexp_out`←`mul_res`
`S_MUL_WAIT`	`mul_done == 0`	`S_MUL_WAIT`	0	0	Wait for `u_modmul`
`S_MUL_WAIT`	`mul_done && exp_cnt>0`	`S_SQ_START`	0	0	`res`←`mul_res`, `exp_shift`≪=1, `exp_cnt--`
`S_MUL_WAIT`	`mul_done && exp_cnt==0`	`S_DONE`	0	1	`res`←`mul_res`, `modexp_out`←`mul_res`
`S_DONE`	*unconditional*	`S_IDLE`	0	0	Return to idle

Table 2: Combinational Mod-P Reduction Truth Table (`modmul.v`)

`acc_shifted >= 2P`	`acc_shifted >= P`	**Output** `acc_next[N+1:0]`
1	X (1)	`acc_shifted - P2_ext`
0	1	`acc_shifted - P_ext`
0	0	`acc_shifted`

Table 3: Operand Multiplexer & Pre-Reduction Control Table

Unit	**Select Condition**	**Selected Input**	**Purpose**
Pre-Reduction MUX	`inp_reg >= P` (1 / 0)	`inp_reg - P` / `inp_reg`	Enforce $0 \leq \text{base} < P$
Operand B MUX	State (`S_SQ_START` / `S_SQ_WAIT`)	`res` / `base`	Square vs. Multiply
`modmul` Bit MUX	`a_reg[N-1]` (1 / 0)	`b_reg` / `0`	Partial product `add_term`

6 Functional Verification

The RTL (`modexp.v`, `modmul.v`) was simulated against the unmodified testbench `modexp_tb.v` and all 10 golden 2048-bit test vectors using **Verilator 5.032**. All **10 / 10 test cases PASSED** with zero failures and zero timeouts.

Table 4: Per-Vector Cycle Count and Verification Summary ($N = 2048$)

Test Vector	inp $< P$	Exp Hamming Weight	Clock Cycles	Status
1	Yes	1005	6,254,594	PASS
2	No (inp $\geq P$)	1001	6,246,402	PASS
3	Yes	982	6,207,490	PASS
4	No (inp $\geq P$)	1082	6,412,290	PASS
5	No (inp $\geq P$)	1016	6,277,122	PASS
6	Yes	1026	6,297,602	PASS
7	Yes	1026	6,297,602	PASS
8	Yes	1021	6,287,362	PASS
9	Yes	1010	6,264,834	PASS
10	Yes	1005	6,254,594	PASS
Average	—	**1017.4**	**6,279,989.2**	**10/10 PASS**

7 ASIC Synthesis, Timing, Power & Energy Results

Synthesis was performed using **Yosys 0.52** and static timing/power analysis using **OpenSTA 3.1.0** with `NangateOpenCellLibrary_typical.lib` (using `BUF_X1` driving cell and `10.0` load constraint).

Table 5: RSA-2048 `modexp` ASIC Implementation Metrics (Nangate 45nm Typical)

Metric	Measured Value
Total Standard Cells	143,730
Total Chip Area	$240,591.41\ \mu m^2$
Sequential Area	$111,277.38\ \mu m^2$ (46.25%)
Combinational Area	$129,314.04\ \mu m^2$ (53.75%)
Clock Cycles (Min / Avg / Max)	6,207,490 / 6,279,989.2 / 6,412,290
Critical Path Delay (T_{min})	20.6742 ns (arrival 20.6355 ns + setup 0.0387 ns)
Maximum Clock Frequency (f_{max})	**48.37 MHz**
Hold Slack (Min Path)	+0.0287 ns (MET)
Power @ 100 MHz (10.0 ns)	43.80 mW (Internal: 29.5 mW, Switching: 9.30 mW, Leakage: 4.99 mW)
Power @ f_{max} (48.37 MHz)	23.76 mW (Dynamic: 18.77 mW, Leakage: 4.99 mW)
Average Latency @ f_{max}	129.83 ms
Energy per RSA-2048 ModExp @ f_{max}	**3.085 mJ**